DOCUMENTO DE DISEÑO DE API — TACOMANDA

Proyecto: Sistema de gestión de comandas del restaurante Tacos Angy.

Fase de Diseño de Software — Apartado 13

Versión documental: 1.0 | Fecha: 23 de octubre de 2026

Estado del proyecto: documentación de contrato OpenAPI y modelo de datos; backend ASP.NET Core pendiente de implementar.

1. PROPÓSITO DEL DOCUMENTO

Documentar cómo se comunicará la interfaz Angular con el futuro backend ASP.NET Core a través de una API REST. Esta definición corresponde al punto 13 de la guía de Diseño de Software. El presente archivo explica los endpoints, las entradas y salidas, los códigos de respuesta y las decisiones de seguridad del contrato.

Tacomanda automatizará operaciones del restaurante Tacos Angy: inicio de sesión, menú, comandas, visualización de cocina, cobros, cierres de caja y retroalimentación. La API es necesaria porque Angular no debe acceder directamente a MySQL; el servidor aplicará las reglas, validará permisos y administrará la persistencia.

2. ARQUITECTURA PREVISTA

Frontend: Angular (features, core, shared) → HTTP/JSON con JWT → ASP.NET Core API (controllers y middleware) → Application (commands, queries, validators y handlers) → Domain (entidades e interfaces), mientras Infrastructure implementa repositorios, EF Core, BCrypt, JWT y acceso a MySQL.

Se adopta como propuesta Clean Architecture con principios SOLID y separación CQRS: los comandos representan cambios de estado y las consultas recuperan vistas de lectura. La implementación, despliegue y prueba de rendimiento se realizarán en una fase posterior.

3. REVISIÓN DE LOS ARCHIVOS ENTREGADOS

Archivo	Contenido verificado	Situación
README.md	Justificación de la API, estado, visualización en Swagger, publicación en GitHub y ejemplos.	Actualizado
docs/swagger.json	Contrato OpenAPI 3.0.3: 16 operaciones HTTP en 7 grupos y esquemas de entrada/salida.	Conservado
database/schema.sql	Creación de base tacomanda y 10 tablas, relaciones y datos iniciales.	Conservado

Archivo	Contenido verificado	Situación
database/seed-empleados.sql	Tres usuarios ficticios de desarrollo con valores BCrypt.	Conservado
docs/flujos-cqrs-crc.pdf	Tarjetas CRC y flujo de login, envío de comanda y cobro.	Conservado
index.html	Página informativa en blanco y negro para GitHub Pages, sin backend.	Añadido

4. CATÁLOGO DE ENDPOINTS

Método	Ruta	Descripción	HTTP previsto
POST	/api/auth/login	Iniciar sesión	200, 400, 401
GET	/api/platillos	Listar platillos	200, 401
POST	/api/platillos	Crear platillo	201, 400, 401
PUT	/api/platillos/{id}	Actualizar platillo	200, 400, 401, 404
DELETE	/api/platillos/{id}	Eliminar platillo	204, 401, 404, 409
GET	/api/mesas	Listar mesas	200, 401
GET	/api/comandas	Listar comandas	200, 401
POST	/api/comandas	Enviar comanda	201, 400, 401, 404
GET	/api/comandas/{id}	Obtener una comanda	200, 401, 404
PUT	/api/comandas/{id}/ estado	Avanzar el estado de una comanda	200, 400, 401, 404, 409
POST	/api/comandas/{id}/ cobrar	Cobrar una comanda	201, 400, 401, 404, 409
GET	/api/pedidos/{id}/ticket	Obtener el ticket de un cobro	200, 401, 404
GET	/api/cierre-caja	Historial de cierres de caja	200, 401
POST	/api/cierre-caja	Registrar cierre de caja	201, 400, 401
GET	/api/retroalimentacion	Historial de calificaciones de la app	200, 401
POST	/api/retroalimentacion	Calificar la app	201, 400, 401

Las operaciones GET son consultas y las operaciones POST, PUT y DELETE corresponden a acciones que pueden modificar información. Algunas rutas como POST /api/comandas/{id}/cobrar representan acciones del negocio.

5. DATOS DE ENTRADA Y RESPUESTAS

5.1 Inicio de sesión

POST /api/auth/login recibe LoginRequest con noControl y contrasena. Ante credenciales correctas define un LoginResponse con token JWT, noControl, nombre, rol y permisos. Los errores previstos incluyen 400 (formato inválido) y 401 (credenciales incorrectas).

5.2 Registro y actualización de comanda

POST /api/comandas utiliza EnviarComandaRequest: mesaNumero y arreglo de items. Cada ítem identifica platillo, cantidad y nota opcional. No se envían precios ni identidad del mesero: el futuro servidor deberá obtener los precios desde la base y el operador desde el JWT.

PUT /api/comandas/{id}/estado recibe EstadoComandaRequest; los estados definidos son abierta, en_preparacion, lista, cerrada y cancelada. El diseño del flujo impide transiciones de cocina inválidas.

5.3 Cobro y ticket

POST /api/comandas/{id}/cobrar recibe tipoPago (efectivo o tarjeta), tipoPedido y montoRecibido. Un cobro creado devuelve un identificador de pedido; el ticket completo se consulta por GET /api/pedidos/{id}/ticket, que contiene monto total y cambio.

5.4 Códigos HTTP y autenticación

Código	Uso previsto
200	Consulta o actualización correcta.
201	Recurso creado o acción registrada.
204	Eliminación sin cuerpo de respuesta.
400	Solicitud con datos inválidos.
401	Credenciales incorrectas o JWT faltante/inválido.
404	Recurso no encontrado.
409	Conflicto con el estado o reglas de negocio.

En el contrato, POST /api/auth/login es público; el resto declara autenticación HTTP Bearer (JWT). La definición detallada de autorización por rol y los códigos 403 aún están pendientes de añadir.

6. DISEÑO DE LA BASE DE DATOS

El script database/schema.sql define diez tablas: empleados, mesas, platillos, productos, platillo_producto, comandas, detalle_comanda, pedidos, encuestas_cierre, retroalimentacion_app. Las relaciones se expresan mediante claves foráneas. Se incluyen registros ilustrativos de mesas,

productos, platillos y sus componentes. El script seed-empleados.sql carga tres cuentas ficticias de desarrollo con hashes BCrypt.

Este esquema puede ejecutarse en un MySQL local. Su existencia no implica que la API ASP.NET Core ya esté conectada. Las credenciales ilustrativas de desarrollo no deben emplearse en producción.

7. FLUJOS CQRS Y TARJETAS CRC

7.1 Autenticación

Angular envía las credenciales; el controlador convierte la petición en un comando, el validador revisa entradas y el manejador consulta al empleado mediante el repositorio. Si la contraseña coincide, un servicio genera JWT y se devuelve la respuesta. Las consultas separadas pueden recuperar permisos para la interfaz.

7.2 Envío de comandas y cocina

El mesero selecciona mesa y platillos; el comando valida cantidades y disponibilidad y guarda comanda y detalles. Cocina consulta el tablero y actualiza los estados permitidos desde abierta hasta lista. Los precios finales se calculan exclusivamente en el servidor.

7.3 Cobro

El cajero consulta comandas listas, envía el comando de cobro, el dominio valida su estado y la política correspondiente (efectivo o tarjeta), y se registra pedido, cierre de comanda y liberación de mesa. El ticket se consulta por un endpoint independiente.

Detalle de clases, responsabilidades y colaboradores: docs/flujos-cqrs-crc.pdf (archivo original del proyecto).

8. CÓMO VISUALIZAR Y ENTREGAR EL CONTRATO

1. Abrir https://editor.swagger.io/.

2. Seleccionar File → Import file y elegir docs/swagger.json.

3. Revisar las 16 operaciones y sus apartados Parameters, Request body, Responses y Schemas.

4. Para entregar, crear un repositorio público Tacomanda y subir el contenido de esta carpeta sin credenciales reales.

5. Opcionalmente habilitar GitHub Pages desde la rama main y raíz del repositorio; esto mostrará index.html como página de documentación. El enlace público no representa una API ejecutable.

9. OBSERVACIONES Y PENDIENTES

A) Identidad: en algunos pasos del PDF CRC anterior aparece número de control en el cuerpo del comando, pero OpenAPI lo obtiene del JWT. Se debe consolidar esta decisión al implementar.

B) Cierre de caja: el JSON actual todavía recibe totalSistema desde el cliente. Es recomendable que la versión definitiva lo calcule en el backend.

C) Roles y cocina: queda por detallar la autorización específica por endpoint y el acceso del personal de cocina, ya que el esquema actual solo define roles admin, mesero y cajero.

D) Falta implementación: no hay controllers ASP.NET Core, handlers, repositorios, conexión activa, pruebas de integración ni API pública ejecutándose en este paquete.

10. CONCLUSIÓN

Los archivos presentados documentan la interfaz prevista entre Angular y ASP.NET Core y satisfacen la finalidad documental de la fase de diseño: describen 16 operaciones, entradas, salidas, autenticación JWT, códigos HTTP, esquema de persistencia y flujos CQRS. La implementación real deberá seguir el contrato o actualizarlo de forma controlada, resolver los puntos pendientes y verificar su funcionamiento mediante pruebas.

Documentos de referencia del proyecto

Contrato OpenAPI: docs/swagger.json. Esquema relacional: database/schema.sql. Datos de prueba: database/seed-empleados.sql. Flujos y CRC: docs/flujos-cqrs-crc.pdf. Instrucciones de revisión: README.md.